

AI-Based Intelligent Search for ERP Systems

Natural-Language Retrieval, Metadata Filtering, and
Exact ERP Path Generation

E-commerce Order Management Prototype

Prepared by: Salmanul Faris

Executive Summary

Project outcome: A hybrid search prototype that converts natural-language ERP questions into structured record filters, semantic matches, and exact breadcrumb-style navigation paths.

The project integrates five e-commerce datasets into a corrected search view containing 89,316 rows and 24 core fields. It combines rule-based intent detection and metadata filtering with all-MiniLM-L6-v2 sentence embeddings, cosine similarity, FAISS retrieval, TF-IDF, and RapidFuzz. Structured queries remain explainable, while paraphrased and product-category searches receive semantic or lexical assistance.

A controlled ten-query test reported accuracy, weighted precision, weighted recall, and weighted F1 of 1.00. These results confirm correctness on the notebook's defined patterns, but the small, vocabulary-aligned test set is not sufficient evidence of production-level generalization.

Source: Executed outputs in the notebook supplied for this report.

CHAPTER 1

1. Project Overview

Measure	Observed value	Meaning
Final search dataset	89,316 × 24	Corrected integrated ERP view
Embedding model	all-MiniLM-L6-v2	384-dimensional query vectors
Product ranking	0.70 TF-IDF + 0.30 fuzzy	Hybrid lexical relevance
Intent evaluation	10 queries; 1.00 metrics	Controlled functional test

1.1 Problem and Objective

Conventional ERP systems require users to know module names, menu hierarchies, field labels, and filter locations. The objective is to let a user enter a plain-English request - for example, 'Show orders that are not delivered' or 'Find toys products' - and receive relevant records or the exact ERP location without knowing the database schema.

1.2 Core Objectives

1. Clean and integrate customer, order, item, payment, and product data.
2. Recognize delivery, amount, payment, customer, product, status, and order intents.
3. Apply transparent metadata filters for precise business constraints.
4. Use semantic and lexical retrieval for paraphrases and product navigation.
5. Return results with relevance information and an exact ERP breadcrumb path.

1.3 Scope

The prototype covers local CSV ingestion, English-language queries, notebook-based experimentation, read-only record retrieval, and inventory-path generation. It does not include authentication, live ERP APIs, transactional updates, multilingual parsing, monitoring, or a deployed user interface.

1.4 Technology Stack

Technology	Project role
pandas / NumPy	Cleaning, joins, aggregation, and filtering
scikit-learn	TF-IDF, cosine similarity, and evaluation metrics
SentenceTransformers	all-MiniLM-L6-v2 semantic embeddings
FAISS	Nearest-neighbour vector retrieval
RapidFuzz	Partial-string product-category matching

CHAPTER 2

2. Dataset and Preparation

2.1 Dataset Composition

The notebook loads five related e-commerce tables. Each raw training table contains 89,316 rows. The external dataset source is not identified in the notebook, so no provider attribution is assumed in this report.

Table	Raw shape	Primary link	Representative content
Customers	89,316 × 4	customer_id	city, state, ZIP prefix
Orders	89,316 × 7	order_id / customer_id	status and timestamps
Order Items	89,316 × 5	order_id / product_id	seller, price, shipping
Payments	89,316 × 5	order_id	type, installments, value
Products	89,316 × 6	product_id	category and dimensions

2.2 Key Preparation Decisions

6. Convert order timestamps to datetime and coerce invalid values to missing.
7. Preserve 1,889 missing delivery timestamps as meaningful 'Not Delivered' records.
8. Standardize missing categories and locations as Unknown and median-impute numeric fields.
9. Aggregate payment information at order level before joining the search view.
10. Create search_text from order, customer, status, product, and payment fields.

Critical merge correction: The initial product join expanded to 2,529,486 rows because Products contained 61,865 duplicate rows. Deduplicating to 27,451 unique product IDs restored the final dataset to 89,316 rows × 24 columns.

CHAPTER 3

3. System Architecture and Search Logic

3.1 Hybrid Architecture

The system uses a hybrid design. Deterministic filters handle constraints that map directly to ERP fields, semantic retrieval handles paraphrases, and a dedicated lexical layer ranks product categories and generates navigation paths.

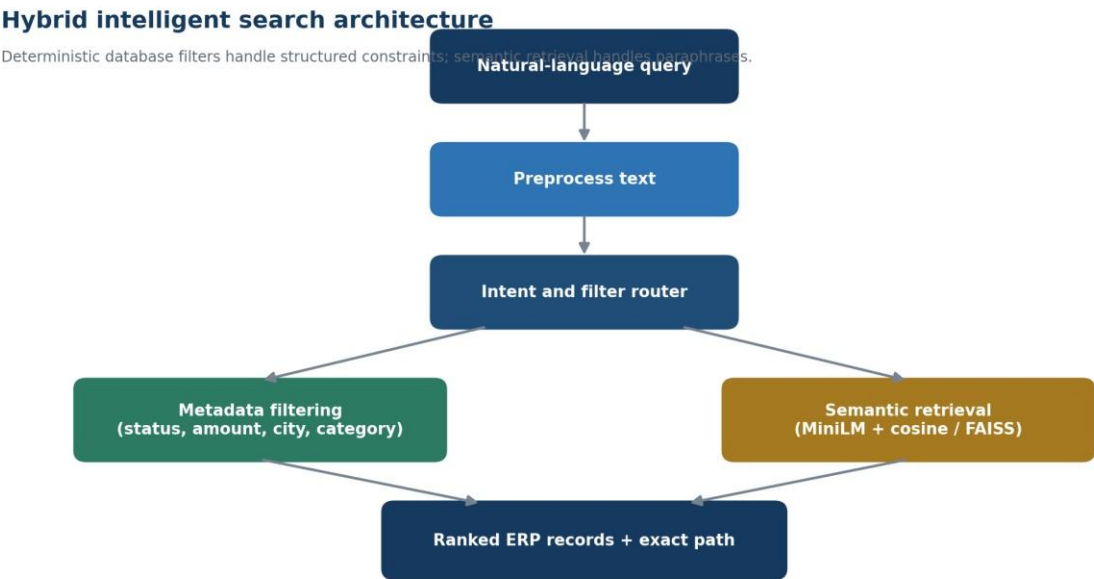


Figure 1. Hybrid query understanding and ERP retrieval architecture.

3.2 Retrieval Layers

Layer	Method	Primary output
Preprocessing	Lowercase, remove special characters, normalize spaces	clean query
Intent routing	Priority-ordered keyword rules	intent label
Metadata search	pandas field filters	matching records

Semantic search	MiniLM + cosine similarity / FAISS	related query examples
Product path	TF-IDF + RapidFuzz	ranked category and ERP path

CHAPTER 4

4. Algorithms and Implementation

4.1 Core Algorithms

The all-MiniLM-L6-v2 model converts each query into a 384-dimensional vector. Cosine similarity ranks labelled examples by semantic direction, while FAISS IndexFlatL2 demonstrates exact nearest-neighbour retrieval. For product navigation, TF-IDF uses unigram and bigram features with sublinear term frequency, and RapidFuzz adds tolerance for partial spelling overlap. **final_score = 0.70 × TF-IDF score +**

0.30 × fuzzy score 4.2 Main Functions

Function group	Responsibility
preprocess_text / detect_intent	Normalize input and identify query intent
amount / payment / city / category filters	Apply exact structured constraints
semantic_search / faiss_search	Return semantically related intent examples
unified_smart_search	Combine direct filtering with semantic fallback
search_erp / create_erp_path	Rank product categories and format the exact path

4.3 Example

```
unified_smart_search("Show orders that are not delivered") search_and_show_path("toys")
```

```
Expected path: ERP > Inventory > Products > Toys
```

Score interpretation: Cosine similarity is higher-is-better, whereas FAISS L2 distance is lower-is-better. These values must not be compared directly without normalization or calibration.

4.4 Streamlit User Interface

AI-Based Intelligent Search for ERP

Search ERP data using natural language

Enter your search query:

Show orders paid by credit card

Search Results

SEARCH QUERY: Show orders paid by credit card

Result 1

Payment ID : Axfy13Hk4PIk
Order ID : Axfy13Hk4PIk
Payment Type : credit_card
Payment Value : 259.14
Exact ERP Path : ERP → Sales → Payments

Result 2

Payment ID : v6px92oS8cLG
Order ID : v6px92oS8cLG
Payment Type : credit_card
Payment Value : 382.39
Exact ERP Path : ERP → Sales → Payments

AI-Based Intelligent Search for ERP

Search ERP data using natural language

Enter your search query:

Show orders that are not delivered

Search Results

SEARCH QUERY: Show orders that are not delivered

Result 1

order_id : P5R6jr1qZdh4
customer_id : FrEvnEIMKGpr
order_status : canceled
order_purchase_timestamp: 2017-07-24 11:38:43
order_approved_at : 2017-07-24 11:50:18
order_delivered_timestamp: nan
order_estimated_delivery_date: 2017-08-07
Exact ERP Path : ERP → Sales → Orders

Result 2

order_id : C21fWds5zL0W
customer_id : iFsA2RrzVaT5
order_status : shipped
order_purchase_timestamp: 2017-02-04 12:58:55
order_approved_at : 2017-02-04 13:10:38
order_delivered_timestamp: nan
order_estimated_delivery_date: 2017-03-15
Exact ERP Path : ERP → Sales → Orders

AI-Based Intelligent Search for ERP

Search ERP data using natural language

Enter your search query:

Show customers from Sao Paulo

Search Results

SEARCH QUERY: Show customers from Sao Paulo

Result 1

customer_id : hCT0x9JiGXBQ
customer_zip_code_prefix: 58125
customer_city : varzea paulista
customer_state : SP
Exact ERP Path : ERP → Customers

Result 2

customer_id : PxA7fv9spyhX
customer_zip_code_prefix: 3112
customer_city : armacao dos buzios
customer_state : RJ
Exact ERP Path : ERP → Customers

AI-Based Intelligent Search for ERP

Search ERP data using natural language

Enter your search query:

toys

Search Results

SEARCH QUERY: toys

Result 1

Product ID : 0vbEvli2JYJu
Category : toys
Relevance Score : 0.24
Exact ERP Path : ERP → Inventory → Products → Toys

Result 2

Product ID : 0vbEvli2JYJu
Category : toys
Relevance Score : 0.24
Exact ERP Path : ERP → Inventory → Products → Toys

CHAPTER 5

5. Testing and Results

5.1 Controlled Evaluation

Ten labelled queries covering customer, product, payment, delivery, status, and order searches were evaluated with scikit-learn. The notebook reports the following results.

Metric	Reported value	Interpretation
Accuracy	1.0000	All ten controlled queries classified correctly
Weighted precision	1.0000	No false-positive intent assignments
Weighted recall	1.0000	No labelled intents missed
Weighted F1	1.0000	Perfect balance on this small test

5.2 Sample Functional Results

Query	Observed result
Orders not delivered	1,889 records
Orders above 5000	8 records
Orders below 100	26,666 records
Credit-card payments	65,814 records
Customers from Sao Paulo	14,352 records
Products in toys category	67,027 records; Toys path

Evaluation caution: The ten handcrafted queries closely match the rule vocabulary. The 100% scores demonstrate functional consistency, not robustness to unrestricted real-world language.

CHAPTER 6

6. Deployment, Security, and Governance

6.1 Recommended Deployment Pattern

11. Move hard-coded file paths and model locations into environment-based configuration.
12. Separate data preparation, intent parsing, retrieval, and path mapping into modules.

13. Load fitted vectorizers, embeddings, and FAISS indexes once at application startup.
14. Expose the search workflow through Streamlit or a read-only REST API.
15. Display interpreted intent, active filters, result count, relevance, and the ERP path. **6.2**

Essential Controls

Risk	Recommended control
Unauthorized record access	Role-based and field-level authorization
Incorrect semantic match	Threshold, explanation, and user confirmation
Join-driven data distortion	Key-uniqueness assertions before every merge
Sensitive data exposure	Mask identifiers; encrypt data in transit and at rest
Model or dependency drift	Pinned versions, audit logs, and regression tests

Business-critical retrieval should remain read-only until the user confirms the target record and operation. Query, user, filter, result count, model version, and timestamp should be logged without storing unnecessary sensitive values.

CHAPTER 7

7. Limitations, Future Scope, and Conclusion

7.1 Current Limitations

16. The semantic query catalogue contains only about 24-25 labelled examples.
17. Rules cover narrow English vocabulary and mainly one intent at a time.
18. FAISS indexes example queries rather than the full ERP record corpus.
19. Category imbalance is severe; toys accounts for 67,027 rows.
20. TF-IDF on raw category labels gives weak matches for synonyms such as watch.
21. No persistent UI, authentication, monitoring, or live database integration is included. **7.2**

Priority Enhancements

Priority	Enhancement	Expected benefit
1	Refactor paths, modules, and dependency versions	Reproducible clean execution
2	Add aliases, multi-filter parsing, and relevance thresholds	Higher query quality
3	Index normalized ERP record text	Search beyond example-query matching
4	Implement Streamlit/API, authentication, and audit logs	Controlled pilot deployment

5	Expand independent evaluation and monitoring	Evidence of real-world robustness
---	--	-----------------------------------

7.3 Conclusion

The notebook is a technically sound educational proof of concept. Its strongest feature is the combination of exact business filters with semantic and fuzzy retrieval. After data-pipeline refactoring, broader evaluation, relevance calibration, security controls, and a user-facing Streamlit or API layer, it can evolve into a practical ERP smart-search service.

Final assessment: Suitable for demonstration after refactoring; not yet ready for unsupervised production use.

Selected References

pandas (pandas.pydata.org/docs); *scikit-learn* (scikit-learn.org/stable); *SentenceTransformers* (sbert.net); *FAISS* (faiss.ai); *RapidFuzz* (rapidfuzz.github.io/RapidFuzz); and the project notebook supplied by the user, accessed August 2026.